

Evaluating PQC KEMs, Combiners, and Cascade Encryption via Adaptive IND-CPA Testing Using Deep Learning

Simon Calderon^{1,2}, Niklas Johansson^{1,2}, and Onur Günlü^{3,1}

¹ Department of Electrical Engineering, Linköping University, Sweden
{simon.calderon, niklas.johansson}@liu.se,

² Sectra Communications, Sweden

³ Lehrstuhl für Nachrichtentechnik, Technische Universität Dortmund, Germany
onur.guenlue@tu-dortmund.de

Abstract. Ensuring ciphertext indistinguishability is fundamental to cryptographic security, but empirically validating this property in real implementations and hybrid settings presents practical challenges. The transition to post-quantum cryptography (PQC), with its hybrid constructions combining classical and quantum-resistant primitives, makes empirical validation approaches increasingly valuable. By modeling indistinguishability under chosen-plaintext attack (IND-CPA) games as binary classification tasks and training on labeled ciphertext data with binary cross-entropy loss, we study deep neural network (DNN) distinguishers for ciphertext indistinguishability. We apply this methodology to PQC key encapsulation mechanisms (KEMs). We specifically test the public-key encryption (PKE) schemes used to construct examples such as ML-KEM, BIKE, and HQC. Moreover, a novel extension of this DNN modeling for empirical distinguishability testing of hybrid KEMs is presented. We implement and test this on combinations of PQC KEMs with unpadded RSA, RSA-OAEP, and plaintext. Finally, methodological generality is illustrated by applying the DNN IND-CPA classification framework to cascade symmetric encryption, where we test combinations of AES-CTR, AES-CBC, AES-ECB, ChaCha20, and DES-ECB. In our experiments on PQC algorithms, KEM combiners, and cascade encryption, no algorithm or combination of algorithms demonstrates a significant advantage (evaluated via two-sided binomial tests with significance level $\alpha = 0.01$), consistent with theoretical guarantees that hybrids including at least one IND-CPA-secure component preserve indistinguishability, and with the absence of exploitable patterns under the considered DNN adversary model. These illustrate the potential of using deep learning as an adaptive, practical, and versatile empirical estimator for indistinguishability in more general IND-CPA settings, allowing data-driven validation of implementations and compositions and complementing the analytical security analysis.

Keywords: Deep Learning for Post-Quantum Cryptography Transition · Hybrid encryption · IND-CPA · Combiners

1 Introduction

Development and validation of new cryptographic algorithms require significant effort. Careful consideration of the security assumptions and rigorous cryptanalysis are needed before an algorithm is employed to secure information. In recent years, the development of new algorithms has been driven by the looming threat of quantum computers running Shor’s algorithm [38], which could be used to efficiently attack asymmetric encryption schemes, such as RSA [33] and Diffie-Hellman [22]. This led the National Institute of Standards and Technology (NIST) [26] to initiate a standardization process for post-quantum cryptography (PQC) algorithms that, in 2024, resulted in the standardization of ML-KEM [28] for key exchange; SLH-DSA [29] and ML-DSA [27] for digital signatures. Due to long replacement times for cryptographic primitives and "harvest now, decrypt later" attacks [24], the transition to PQC is currently of high priority [14].

One technique used to facilitate the adoption of PQC is called hybrid encryption [8]. Hybrid methods combine several existing algorithms in such a way that the combined algorithm is secure as long as at least one of the component algorithms is secure. This allows PQC and classical algorithms to be combined to benefit from the cryptanalysis performed and security assumptions established for each algorithm, thus facilitating faster adoption by spreading the risk. Such hybrid methods can, in principle, also be used to combine multiple different PQC algorithms without any classical component, to provide similar risk mitigation in scenarios where the classical algorithms have been compromised.

We remark that the connections between cryptography and machine learning methods were mentioned as early as 1991 by Rivest [32], drawing a parallel between machine learning and cryptanalysis as two cases in which one tries to learn an unknown function. Since then, there have been a number of applications of machine learning to cryptography and cryptanalysis. For instance, Alani [3] developed a NN-based cryptanalysis attack on the Data Encryption Standard (DES) [25] and 3DES [6], which are classical cryptographic methods. The secret keys were extracted using only 2^{11} and 2^{12} plaintext-ciphertext pairs, respectively. Gohr used NNs to create neural distinguishers on reduced round Speck [17]. Baksi et al. expanded on Gohr’s work, demonstrating its applicability to reduced round versions of other lightweight ciphers such as Ascon [5]. Moreover, Volpe and Gauthier-Umaña [40] tested the IND-CPA property of HQC.pke [2] using a k-nearest neighbor (kNN) approach that classifies ciphertexts based on the distance to ciphertexts with known plaintext. Using messages specially targeted for HQC, the authors were able to distinguish between ciphertexts with approximately 80% accuracy, indicating that HQC.pke might not be IND-CPA. Recently, Kim et al. [21] demonstrated the use of DNNs for cryptanalysis, which includes modeling IND-CPA games as binary classification tasks for a DNN. We extend their method to establish deep learning-based IND-CPA results on both PQC Key Encapsulation Mechanisms (KEMs) and hybrid methods that combine PQC and classical cryptographic methods, thereby providing an adaptive testing method for complex hybridization methods.

1.1 Main Contributions

This paper extends a DNN-based approach used for the IND-CPA testing, originally proposed in [21], to demonstrate its applications in the testing of PQC KEMs, and further extends it to test hybrid KEMs. The proposed neural-network framework complements formal security analysis by providing a practical and flexible, empirical tool to evaluate indistinguishability and identify potential weaknesses in implementations and composed constructions.

The main contributions of this work include:

- We use a DNN-based modeling of the IND-CPA game to empirically test the IND-CPA properties of the underlying public key encryption (PKE) schemes used in ML-KEM, BIKE, and HQC.
- We present a novel extension to this framework to test hybridized KEMs. Specifically, our extension allows for testing hybridized KEMs, where the KEM is constructed using a provably-secure combiner of the form $k = F(k_1, c) \oplus F(k_2, c)$ [16], where $\oplus$ is bitwise XOR, k_1 and k_2 are shared secrets from the component KEMs, c is the concatenated ciphertext, F is some function, and k is the resulting shared secret, by combining any KEM with any asymmetric cipher, such as RSA.
- We test and demonstrate the methodological generality of our deep learning-based constructions by applying them also to other hybridized scenarios, such as cascade symmetric encryption [36].
- For the experiments above, we introduce rigorous statistical analysis, unlike existing deep learning-based results, through two-sided binomial hypothesis testing to determine whether the neural binary classifiers gain a non-negligible advantage or not.

1.2 Paper Outline

The remainder of this paper is organized as follows. Section 2 introduces our notation, some relevant information theoretic and cryptographic definitions, and the main security notions that are used throughout this work. Section 3 introduces the deep neural network modeling of IND-CPA games, our selection of cryptographic algorithms, and our DNN training setup. In Section 4, we present our results from applying our DNN modeling to our selection of algorithms and algorithm combinations. Finally, Section 5 concludes the paper.

2 Preliminaries

In this section, we present the notation used throughout the paper and provide basic definitions. We also cover the hybrid encryption methods that are tested for IND-CPA in this work.

2.1 Notation

Unless otherwise noted, we will use the following notation. Denote plaintext messages by m , ciphertexts by c , cryptographic keys by k , and an asymmetric key pair by (pk, sk) , consisting of a public key pk and a secret key sk . $x \xleftarrow{\$} S$ denotes the uniform sampling of x from a set S . We define $[a : b] := \{n \in \mathbb{N} | a \leq n \leq b\}$. We use θ to denote the parameters of a DNN. For DNN training, we denote the actual class of the sample i by Y_i and the network's predicted output by $\hat{Y}_i$.

2.2 Basic Definitions

In this section, we present the definitions of the mechanisms and analysis methods used throughout the paper.

Definition 1 (Negligible function [18, p. 16]). *We call a function $\varepsilon : \mathbb{N} \rightarrow \mathbb{R}$ negligible, if for every positive polynomial $p(x)$ there exists an N such that for all $n > N$, we have*

$$\varepsilon(n) < \frac{1}{p(n)}. \quad (1)$$

Definition 2 (Public key encryption schemes (PKEs) [11]). *For a security parameter n , a PKE scheme Π is defined as a set of three algorithms:*

- $\Pi.\text{keygen}(n) \rightarrow (pk, sk)$
- $\Pi.\text{enc}(pk, m) \rightarrow c$
- $\Pi.\text{dec}(sk, c) \rightarrow m$.

We now define the main security notion used. While the following definition is formulated for PKEs, the corresponding definition for symmetric schemes is analogous (under appropriate modifications) and is omitted for brevity.

Definition 3 (Chosen plaintext experiment [20, p. 74]). *For an encryption scheme Π , an adversary $\mathcal{A}$ and a security parameter n , the experiment $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n)$ is defined by the following game: A challenger $\mathcal{C}$ uses $\Pi.\text{keygen}(n)$ to generate a key pair (pk, sk) and gives pk to the adversary $\mathcal{A}$. The adversary then selects two messages (m_0, m_1) of equal length and gives them to the challenger. The challenger proceeds by selecting a bit b uniformly at random and handing the adversary $\Pi.\text{enc}(pk, m_b)$. The adversary is tasked with outputting a bit b' . The output of the experiment is 1 if $b = b'$ and 0 otherwise.*

Definition 4 (Indistinguishability under chosen plaintext attack (IND-CPA) [20, p. 75]). *An encryption scheme Π has indistinguishable ciphertexts under chosen plaintext attack if for all probabilistic polynomial-time adversaries $\mathcal{A}$ there exists a negligible function $\varepsilon(n)$ such that*

$$\Pr \left[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n) = 1 \right] \leq \frac{1}{2} + \varepsilon(n). \quad (2)$$

We now define information-theoretic notions used in this work.

Definition 5 (Shannon entropy [12, p. 13]). Let P be a distribution on a finite (or countable) set $\mathcal{X}$, and let p denote the probability mass function associated with P . That is, if X is a random variable distributed according to P , then $P[X = x] = p(x)$. The entropy of X (or of P) is defined as

$$H(X) := - \sum_{x \in \mathcal{X}} p(x) \log p(x). \quad (3)$$

Definition 6 (Kullback-Leibler divergence [12, p. 14]). Let P and Q be distributions defined on a discrete set $\mathcal{X}$. Then the Kullback-Leibler (KL) divergence between them is

$$D_{\text{KL}}(P||Q) := \sum_{x \in \text{supp}(p_x)} p(x) \log \frac{p(x)}{q(x)} \quad (4)$$

where $\text{supp}(\cdot)$ refers to the support of a probability distribution.

Definition 7 (Binary cross entropy (BCE) [19]). Let $P(x)$ and $Q(x)$ be distributions on the set $\{0, 1\}$ with $p = P(1)$ and $q = Q(1)$, then the BCE is defined as:

$$\text{BCE}(P, Q) = \mathbb{E}_{x \sim P}[-\log(Q(x))] = -p \log(q) - (1 - p) \log(1 - q) \quad (5)$$

The BCE can also be expressed using KL-divergence as

$$\text{BCE}(P, Q) = H(P) + D_{\text{KL}}(P||Q). \quad (6)$$

As can be seen in (6), for a fixed distribution P , $\text{BCE}(P, Q)$ is minimized by minimizing the Kullback Leibler divergence between P and Q for fixed $H(P)$.

The BCE can be used to quantify the discrepancy between the true labels and the approximated distributions produced by the network. Specifically, the average BCE over a batch of size N is used. Let $Y_i \in \{0, 1\}$ be the actual label of sample i , and $\hat{Y}_i \in (0, 1)$ be the probability $\Pr[Y_i = 1]$ predicted by the network, then the batch loss is given by

$$\text{BCE}(Y, \hat{Y}) = -\frac{1}{N} \sum_{i=1}^N [Y_i \log(\hat{Y}_i) + (1 - Y_i) \log(1 - \hat{Y}_i)]. \quad (7)$$

The BCE loss function can be interpreted as the expected code length (in, e.g., bits per label) when encoding true labels using the network's predicted probabilities. It is also a differentiable function, allowing for the use of the stochastic gradient descent (SGD) method [34] to optimize the weights of the DNN.

2.3 Hybrid Encryption

Combining cryptographic algorithms dates back to Shannon [37]. Several potential goals motivate such combinations: creating stronger encryption than single

algorithms, hedging against potential weaknesses in any individual algorithm, and providing fail-safes against misconfigurations. For instance, the National Security Agency (NSA)'s Commercial Solutions for Classified (CSfC) framework recommends layered encryption for data at rest (i.e., stored data) to prevent security loss due to misconfigurations [30].

a) Hybridized KEMs We first consider the hybridization of KEMs.

Definition 8 (KEM Combiners [16, p. 9]). *Let $k_1, \dots, k_n$ be the output from n KEMs, and let $c_1, \dots, c_n$ be their corresponding ciphertexts. A KEM combiner of n KEMs is a function W such that*

$$k = W(k_1, \dots, k_n, c_1, \dots, c_n) \quad (8)$$

where k is the output shared secret.

Note that while Definition 8 defines KEM combiners to combine the outputs of arbitrarily many KEMs, for the rest of this work, we focus exclusively on combiners that combine two KEMs. More specifically, we limit the scope to combiners of the form $k = F(k_1, c) \oplus F(k_2, c)$, as defined next.

Definition 9 (Provably-secure combiners [16, p. 26]). *Let $k_1, k_2 \in \mathcal{K}$ be the output shared secrets from two KEMs, $c_1, c_2 \in \mathcal{C}$ be their corresponding ciphertexts. Define $c = c_1 || c_2$, and let $F : \mathcal{K} \times \mathcal{C} \rightarrow \mathcal{K}$ be some function. We have*

$$k = F(k_1, c) \oplus F(k_2, c) \quad (9)$$

where k is the output shared secret. The combiner in (9) is provably-secure against chosen-plaintext attacks if F is a pseudo-random function (PRF) in the standard model [16].

A special case of a combiner of the form in (9) is the XOR-combiner with $W(k_1, k_2, c_1, c_2) = k_1 \oplus k_2$. Using the XOR-combiner yields an IND-CPA KEM, provided that at least one of the component algorithms is IND-CPA [16].

b) Hybridization of Symmetric Encryption Similar to KEMs, symmetric encryption algorithms can be combined in various ways. Next we focus on cascade encryption.

Definition 10 (Cascade encryption [36]). *For two symmetric encryption algorithms Enc_a and Enc_b , cascade encryption is defined as*

$$\text{Enc}_{\text{cascade}}(m) = \text{Enc}_a(\text{Enc}_b(m)). \quad (10)$$

Note that the resulting cipher of cascade encryption may depend on the order in which the algorithms are applied, except for the case where both algorithms are XOR-based synchronous stream ciphers (such as AES-CTR or ChaCha20), for which the resulting cipher will be the same regardless of order. Moreover, we refer to Enc_a as the outer cipher and Enc_b as the inner cipher.

3 Deep Learning, KEM, and Cascade Encryption Setups for IND-CPA Testing

In this section, we introduce the DNN models for testing IND-CPA and then apply them to the PKE schemes used to construct PQC KEMs, such as ML-KEM, BIKE, and HQC. We then present our extended algorithm for testing the IND-CPA of hybrid methods using combiners of the form $k = F(k_1, c) \oplus F(k_2, c)$, as described in Definition 8. We also give our selections of algorithms and algorithm combinations to be tested, motivate our selections of network hyperparameters used for our experimental evaluations, and describe our application of DNN modeling to cascade encryption. Finally, we describe our statistical evaluations of the DNN performance.

3.1 The IND-CPA Testing Algorithm for Non-hybridized KEMs and Cascade Encryption

We employ Algorithm 1 from [21] to model IND-CPA games using DNNs. For each KEM, a DNN is trained to distinguish between two classes of ciphertext, one class consisting of encryptions of uniformly random plaintexts, and another class consisting of encryptions of a fixed plaintext, in this case, all 0s. Each DNN is trained using stochastic gradient descent using the BCE as the loss function. If the encryption scheme used is IND-CPA secure, the resulting DNN should not be able to classify ciphertexts with an accuracy better than random guessing.

We remark that Algorithm 1 is not directly applicable to all KEMs, especially KEMs constructed with Fujisaki-Okamoto transform or variants thereof, since Algorithm 1 requires selecting a plaintext, which is not possible for such constructions. However, Algorithm 1 can be applied to test the IND-CPA security of the PKE schemes that are used in the Fujisaki-Okamoto style transform to create the KEM. The motivation to test this is that the IND-CPA security of the underlying PKE is used to prove the IND-CCA2 security of the KEM [15]. Moreover, Algorithm 1 is also used for IND-CPA testing of cascade symmetric algorithms.

Algorithm 1 IND-CPA BCE Classification adversary [21]

Input Plaintext set $X_0 \xleftarrow{\$} [0 : 2^{128} - 1]^N$ to challenger.
Input Plaintext set $X_1 \leftarrow \{0^\ell\}^N$ to challenger.
Challenger outputs ciphertexts sets Y_0 and Y_1 from X_0 and X_1
Label ciphertexts Y_0 with "0" for all in set, and ciphertexts Y_1 with "1" for all in set
Form dataset Y from Y_0 and Y_1 .
Initialize network parameters θ .
Repeat until convergence:
 Find **BCE**($Y, \hat{Y}$)
 Compute SGD optimizing and updating θ

3.2 The IND-CPA Testing Algorithm for KEMs using a combiner of the form $k = F(k_1, c) \oplus F(k_2, c)$

For scenarios in which a combiner of the form $k = F(k_1, c) \oplus F(k_2, c)$ (for example, the XOR combiner or the provably secure combiner) is used, we apply Algorithm 2. In these experiments, the DNN receives as input the concatenation of the ciphertext outputs of the two components. The corresponding binary classification task distinguishes between (i) samples where the two ciphertext components decrypt/decapsulate to the same underlying value (the KEM shared secret), and (ii) samples where they decrypt/decapsulate to two independent values of the same length, generated independently of each other.

Algorithm 2 IND-CPA BCE Classification adversary for hybrid KEM

Input Shared secret set X_0 , and ciphertext set Y_{A0} from N encapsulations with the KEM.
Input Ciphertext set Y_{A1} from N encapsulations with the KEM.
Input plaintext set $X_1 \xleftarrow{\$} [0 : 2^{sk} - 1]^N$, where k is the KEM shared secret length in bytes.
Challenger outputs ciphertexts sets Y_{B0} and Y_{B1} from X_0 and X_1 using the asymmetric cipher
Challenger forms joint ciphertexts $Y_0 = Y_{A0} || Y_{B0}$ and $Y_1 = Y_{A1} || Y_{B1}$ by pairwise concatenating elements
Label ciphertexts Y_0 with "0" for all in set, and Y_1 with "1" for all in set
Form dataset Y from Y_0 and Y_1 .
Initialize network parameters θ .
repeat until convergence
 Find $\text{BCE}(Y, \hat{Y})$
 Compute SGD optimizing and updating θ

The rationale behind Algorithm 2 is the following XOR identity. Let $F : \mathcal{K} \times \mathcal{C} \rightarrow \{0, 1\}^\ell$ and fix any $c \in \mathcal{C}$. Define

$$\Delta(c) \triangleq F(k_1, c) \oplus F(k_2, c). \quad (11)$$

If $k_1 = k_2$, then $\Delta(c)$ is the all-zero string, as we have $\Delta(c) = F(k_1, c) \oplus F(k_1, c) = 0^\ell$. When k_1 and k_2 are sampled independently and uniformly from $\mathcal{K}$, equality occurs with probability

$$\Pr[k_1 = k_2] = \frac{1}{|\mathcal{K}|}, \quad (12)$$

and for $k_1 \neq k_2$, the XOR identity does not force $\Delta(c)$ to be equal to 0^ℓ . This creates two structurally different cases, which extends and is analogous to the IND-CPA modeling viewpoint in [21], where a classifier is trained to separate two label classes corresponding to different underlying conditions.

3.3 Considered KEMs

To demonstrate the applicability of the deep learning-based IND-CPA testing approach to important and recent KEMs, we select the following algorithms from the submissions to the NIST PQC standardization contest. 1. **ML-KEM** (formerly called CRYSTALS-Kyber) [9] is a lattice-based KEM and the primary PQC KEM selected for standardization by NIST [31]. 2. **HQC** [2] (Hamming Quasi Cyclic codes) is a code-based KEM, selected by NIST for standardization [39]. 3. **BIKE** [4] is also a code-based KEM. While not selected for standardization, it was a contender and has not been broken. For our purposes, we use the instances targeting NIST security level 1, that is, ML-KEM-512, HQC-128, and BIKE-L1.

Since the KEMs listed above do not provide direct chosen-plaintext encryption of user-selected messages, we apply Algorithm 1 to the underlying PKE component used in the KEM construction. Moreover, for schemes targeting chosen-ciphertext security at the KEM level, IND-CCA2 security implies IND-CPA security. We emphasize that this implication concerns the KEM-level security notion, whereas Algorithm 1 is applied to the underlying PKE component, where IND-CPA distinguishing experiments are naturally defined.

3.4 Considered Hybrid KEM Setups

For the testing of the scenario of using Algorithm 2 to test combinations of a KEM with a cipher, we select the same KEMs from the previous section, as they are prominent PQC KEMs considered in the NIST standardizations.

To be able to apply Algorithm 2, we require the second algorithm to be a cipher in the sense that we can set the plaintext. Since the scenario is key exchange, an asymmetric cipher is selected, in this case, RSA. Using RSA allows for testing on how the three PQC candidates perform combined with an IND-CPA algorithm, such as RSA using optimal asymmetric encryption padding (OAEP) [23], and how they combine with a non-IND-CPA algorithm, such as plain RSA without using any padding.

We also test combining with plaintext, that is, concatenating the shared secret or a random bit sequence of equal length to the KEM ciphertext. This is done to test a worst-case scenario, where the cipher is insecure.

Finally, we also test unpadded RSA combined with plaintext, with the purpose of testing a combination of two insecure algorithms. An adversary with access to the public key could use this to encrypt the plaintext and thereby know with certainty if the concatenated plaintext is the same as the one used for RSA.

3.5 Considered Cascade Encryption Setups (Symmetric Methods)

For the cascading combiner, three modes of AES are selected, namely counter-mode (CTR), electronic codebook (ECB), and cipher block chaining (CBC) [13]. Alongside the three AES variants, the stream cipher ChaCha20 [7], and DES in

Table 1: The selection of combinations for DNN classification of cascade combiner ciphers. The rows correspond to the inner cipher, the columns to the outer cipher. X denotes that the combination is tested.

	AES-CBC	AES-CTR	AES-ECB	ChaCha20	DES
AES-CBC		X	X	X	X
AES-CTR	X		X		X
AES-ECB	X	X		X	
ChaCha20	X	X	X		X
DES	X	X		X	

ECB mode are added [25]. This selection contains both algorithms that are IND-CPA (AES-CBC, AES-CTR, ChaCha20) and those that are not (AES-ECB, DES-ECB). We consider two stream ciphers, three block ciphers, and include some of the most popular symmetric encryption schemes.

The rationale for the selection of combinations in Table 1 is to test every combination of outer and inner cipher possible, with a few exceptions: (i) No cipher is tested combined with itself, (ii) The combination of AES-ECB and DES-ECB is not tested since it produces a deterministic cipher, and (iii) Only one of the combinations of AES-CTR and ChaCha20 is included since they are both synchronous XOR-based stream ciphers, and therefore commute with each other.

3.6 Neural Network Architecture, Parameter Selection, and Evaluations

For all algorithms, KEM combinations, and cascade encryptions tested, two networks are trained for each – a small and a big network. Details for these networks are given in Table 2. Moreover, each byte of the ciphertext is used as an input feature to the DNN.

For training data, 500,000 samples per class are generated, and 100,000 samples per class are used as validation data to track network progress and facilitate early stopping. To avoid overfitting the training or validation data, another 100,000 samples are generated to test the final performance of the model. The training data is generated with the same keying material, i.e., the same two keys for the cascade encryption and two key pairs for the KEM combiner.

Both networks used ReLU, which is the activation function $g(z) = \max\{0, z\}$ [19], in the intermediate layers and a sigmoid activation function in the output layer. We train for at most 1000 epochs (the same number used by Kim et al. in [21], 5 times longer than the setup used by [17], and 50 times longer than [5]). To determine this value, we train the network for a sufficiently long time to mitigate overfitting, while using early stopping with a patience parameter of 100, representing 10% of our maximum training budget of 1000 epochs. We only keep the models that perform the best on the validation data. We use a minimum improvement of $\delta_{ES} = 10^{-6}$ (large enough to be used with 32-bit float accuracy),

Table 2: Overview of the two different DNN setups used for each scenario.

	Small Network	Big Network
Number of intermediate layers	2	4
Nodes in intermediate layers	100	600

to account for very subtle learnable patterns. The training is done with a batch size of 1024 samples per batch, allowing for the DNNs to possibly learn weak patterns, if such patterns exist. We use an initial learning rate of 10^{-4} combined with plateau-based learning rate reduction (ReduceLROnPlateau), a common training heuristic where the learning rate is reduced by a factor 0.5 if the loss function does not improve more than $\delta_{LR} = 10^{-5}$ over 20 consecutive epochs [10]. Using this patience allows for the learning rate to be decreased up to five times before early stopping is activated. We also bound the minimal learning rate to 10^{-7} . Both the learning rate reduction and the early stopping are evaluated on the BCE loss function.

Furthermore, we use a momentum of 0.9 and use Nesterov Accelerated Gradient (NAG) [19], Glorot uniform weight initialization [19], and initialize the biases to zero. The training is performed on one Nvidia T4 GPU, with training times for the different network setups ranging from 1 to 3 hours.

3.7 Statistical testing

To test if the DNNs perform better than random guessing, we use a two-sided binomial test, defined below, to test the null hypothesis $H_0 : \pi = 0.5$ against the alternative hypothesis $H_1 : \pi \neq 0.5$, where π is the classification accuracy of the network. The rationale for this is that each validation X_i constitutes a Bernoulli trial with accuracy π .

Definition 11 (Two-sided binomial test [1, p. 13]). *Let X_i denote the outcome of the i -th trial, where X_i follows a Bernoulli distribution with success probability π . For n trials $K = \sum_{i=1}^n X_i \sim \text{Bin}(n, \pi)$ is defined. Let k be the number of correct classifications, and let $\mathcal{I} = \{i : \Pr[K = i] \leq \Pr[K = k]\}$. $H_0 : \pi = \pi_0$ is rejected at significance level $\alpha = 0.01$ if we have*

$$p = \sum_{i \in \mathcal{I}} \binom{n}{i} \pi_0^i (1 - \pi_0)^{n-i} < 0.01. \quad (13)$$

We use the significance level $\alpha = 0.01$, as it is a common significance level used in cryptographic applications, see, e.g., [35, p. 1-4].

4 Results

In this section, we present the results of DNN testing of IND-CPA. Section 4.1 covers the results applied to KEMs, while Sections 4.2 and 4.3 contain the results from testing of hybrid KEMs and cascade symmetric encryption, respectively.

4.1 Results of IND-CPA Testing for KEMs

We apply Algorithm 1 with the network parameters described in Section 3.6 to the algorithms listed in Section 3.3. The resulting classification accuracies for the DNN training can be found in Table 3, and the training history for validation accuracies can be seen in Fig. 1.

Table 3: Accuracies and p-values for the application of Algorithm 1 to single (i.e., non-hybridized) algorithms.

Cryptographic algorithm	Small Network		Big Network	
	Accuracy	p-value	Accuracy	p-value
Plain RSA*	100%	$2^{-199,999}$	100%	$2^{-199,999}$
RSA-OAEP	50.02%	0.86	49.90%	0.37
ML-KEM	50.10%	0.38	50.04%	0.72
BIKE	49.94%	0.57	50.05%	0.65
HQC	50.26%	0.02	50.06%	0.61

*p-value calculated analytically.

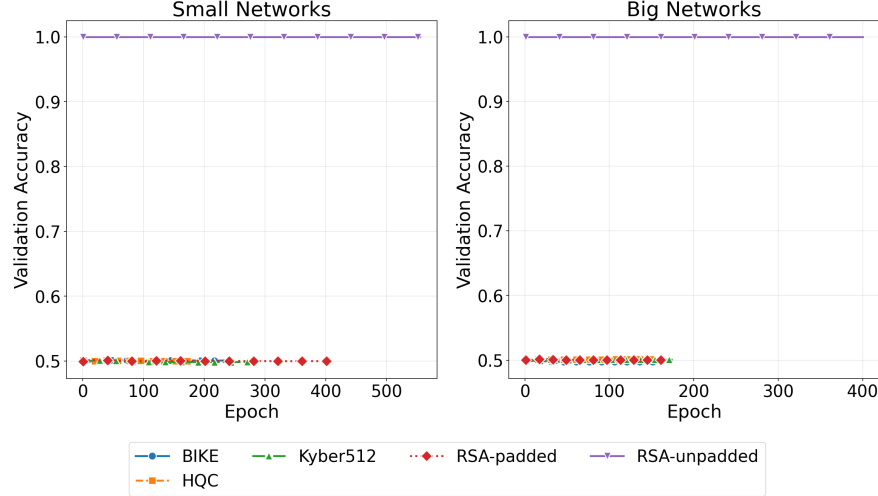


Fig. 1: The validation accuracies plotted over training epochs for the KEMs encryption scenario. The validation accuracy is computed on the validation dataset.

First, we note that both networks achieve a 100% accuracy in classifying the plain RSA ciphertexts, demonstrating complete distinguishability of ciphertexts.

The perfect accuracy is expected, as deterministic (textbook) RSA is well-known to be vulnerable to IND-CPA attacks due to its deterministic encryption. Applying the neural classifier to RSA-OAEP, on the other hand, achieved performance that is statistically indistinguishable from random guessing, which is consistent with RSA-OAEP’s proven IND-CPA security; see also the results in [21].

For BIKE and ML-KEM, we see that both DNNs perform close to 50% in accuracy, with p-values much larger than 0.01, failing to reject the null hypothesis that the networks perform no different than random guessing. This is consistent with the algorithms being IND-CPA.

For HQC, the small network has slightly higher accuracy in classifying ciphertexts, but not enough to constitute a statistically significant advantage over random guessing at the significance level of $\alpha = 0.01$ due to the p-value of 0.02. This advantage is not present in the big networks’ classification accuracy.

Moreover, as observed from Fig. 1, both big and small DNNs perform relatively similarly, achieving 100% accuracy for plain RSA almost directly in terms of the epoch numbers, and not learning any strategy that can classify the ciphertexts with any advantage.

4.2 Results of IND-CPA Testing for KEMs Combined with an asymmetric cipher

Applying Algorithm 2 with DNN parameters described in Section 3.6 to the combination of algorithms described in Section 3.4 yields the classification accuracies given in Table 4. The corresponding training history of validation accuracies is depicted in Fig. 2.

Table 4: Accuracies and p-values for the application of Algorithm 2 to KEMs combined with asymmetric ciphers using combinators of the form $k = F(k_1, c) \oplus F(k_2, c)$.

KEM	Asymmetric cipher	Small Network		Big Network	
		Accuracy	p-value	Accuracy	p-value
ML-KEM	RSA OAEP	49.93%	0.51	50.22%	0.54
ML-KEM	Plain RSA	50.00%	0.97	49.87%	0.25
ML-KEM	Plaintext	50.00%	1.00	49.73%	0.01
HQC	RSA OAEP	50.16%	0.16	50.15%	0.17
HQC	Plain RSA	49.88%	0.28	50.21%	0.06
HQC	Plaintext	50.15%	0.17	50.00%	0.82
BIKE	RSA OAEP	50.10%	0.36	50.20%	0.08
BIKE	Plain RSA	50.26%	0.02	50.00%	0.94
BIKE	Plaintext	50.06%	0.60	49.95%	0.64
Plain RSA	Plaintext	50.12%	0.28	49.86%	0.22

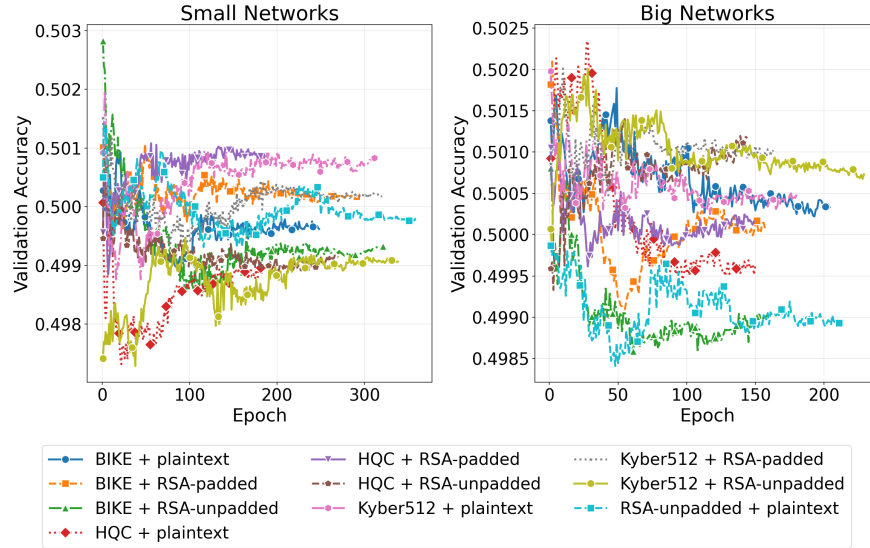


Fig. 2: The validation accuracies computed on the validation dataset vs. training epochs for hybridized KEMs.

The results in Table 4 show the robustness of the provably-secure combiner, as the neural classifier does not learn any patterns that allow it to classify the ciphertexts with any accuracy better than random guessing. Notably, even when an insecure component algorithm, such as plain RSA or plaintext, is used, the observed accuracies remain close to 50% under the considered DNN adversary model. For constructions covered by the combiner theory from [16], as discussed in Section 2.3, this is consistent with the theoretical guarantee that security is inherited from the stronger component. Only two results deviate somewhat more from 50% accuracy, namely BIKE + Plain RSA (small network: 50.26%, $p = 0.02$) and ML-KEM + plaintext (big network: 49.73%, $p = 0.014$). However, these deviations do not constitute a statistically significant deviation from 50% at the significance level $\alpha = 0.01$.

One interesting combination here is that of Plain RSA combined with plaintext. An adversary tasked with classifying the hybrid ciphertext $c_{\text{RSA}}||m$, who also has access to an RSA encryption oracle, could encrypt the plaintext m to determine if it is equal to c_{RSA} . By doing this, the adversary could determine the class of the hybrid ciphertext with perfect accuracy. However, our DNN adversary operates under a more restrictive model, as it lacks access to an encryption oracle. This highlights the fact that the ciphertext distinguishability can depend on other factors, such as knowledge of public parameters or access to an encryption oracle. This result does not imply security for this pair of algorithms; rather, it highlights a limitation of using this DNN approach for IND-CPA analysis

4.3 Results of IND-CPA Testing of Cascade Encryption

For the case of cascade encryption, when we apply Algorithm 1 and the network parameters from Section 3.6 to the algorithm selection described in Section 3.5, we obtain the validation accuracies depicted in Table 5 for the small network setup, and Table 6 for the big network setup. The history of validation accuracies is depicted in Fig. 3.

Table 5: The test validation accuracies and p-values, within the small network setup, for tests on cascade symmetric encryption. The rows correspond to the inner cipher, the columns to the outer cipher.

	AES-CBC	AES-CTR	AES-ECB	ChaCha20	DES
AES-CBC		50.02% (0.82)	50.10% (0.37)	50.19% (0.10)	50.01% (0.92)
AES-CTR	49.97% (0.80)		49.90% (0.35)		50.08% (0.48)
AES-ECB	49.85% (0.18)	50.21% (0.06)		49.88% (0.30)	
ChaCha20	50.06% (0.61)	49.99% (0.93)	49.99% (0.92)		49.99% (0.95)
DES	50.08% (0.50)	49.75% (0.03)		50.02% (0.83)	

Table 6: The test validation accuracies and p-values, within the big network setup, for tests on cascade symmetric encryption. The rows correspond to the inner cipher, the columns to the outer cipher.

	AES-CBC	AES-CTR	AES-ECB	ChaCha20	DES
AES-CBC		50.00% (0.97)	50.16% (0.14)	49.85% (0.17)	50.27% (0.02)
AES-CTR	50.05% (0.68)		50.11% (0.32)		50.02% (0.86)
AES-ECB	49.98% (0.88)	50.14% (0.20)		49.74% (0.02)	
ChaCha20	50.18% (0.11)	49.98% (0.86)	50.02% (0.86)		49.97% (0.78)
DES	50.05% (0.63)	50.01% (0.90)		49.85% (0.18)	

The results shown in Tables 5 and 6 empirically validate the ciphertext indistinguishability of cascade symmetric ciphers, as the neural classifier fails to classify the ciphertexts with any accuracy better than random guessing. The 17

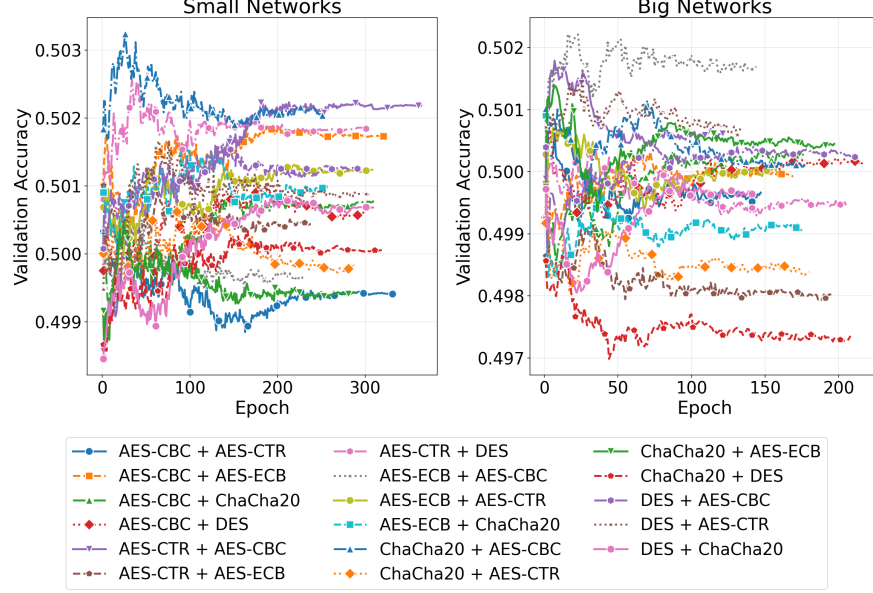


Fig. 3: The validation accuracies plotted over training epochs for the cascade encryption scenario. The validation accuracy is computed on the validation dataset.

combinations tested yield accuracies in the range 49.74%-50.27%. Notably, the combinations remain resistant to the DNN classifier when the cascading is done with one deterministic non-IND-CPA cipher, such as AES-ECB or DES.

5 Conclusion and Future Work

In this work, we applied the DNN IND-CPA classification framework to validate the IND-CPA property of the underlying PKE components used to construct three PQC algorithms, namely ML-KEM, HQC, and BIKE. Our results demonstrated that no classifier achieves an accuracy that is significantly different from random guessing, which is consistent with the algorithms being IND-CPA. Furthermore, we presented an extension of the framework to evaluate ciphertext distinguishability for hybrid constructions using a combiner of the form $k = F(k_1, c) \oplus F(k_2, c)$ [16] under the considered DNN adversary model. We also demonstrated that the methodology of using a DNN to model IND-CPA games can also be applied to cascade encryption. Our experiments provided no evidence that cascade encryption degrades IND-CPA security, because in all tested configurations, the classifier achieved accuracy indistinguishable from random guessing whenever at least one component algorithm is IND-CPA.

For all of our experiments, we have applied a rigorous statistical analysis method to validate that the accuracy of our trained models is statistically indistinguishable from that of random guessing. Our results demonstrate the ver-

satility in using a DNN classifier to validate the IND-CPA security of both individual algorithms and composite constructions, including hybrid KEMs and cascade encryption.

We also discussed that there are some limitations of the DNN IND-CPA classification framework due to the differences between the standard IND-CPA threat model and the threat model for the neural classifier, as the neural classifier does not have access to public keys or encryption oracles. While this poses some limits on what attack vectors the DNN modeling can capture, the model still captures various fundamental aspects of the IND-CPA game. Moreover, a neural classifier that scores significantly better than random guessing will strongly indicate that the algorithm tested is not IND-CPA. For future work, it will therefore be of high interest to develop neural classifiers that can incorporate other factors, such as the encryption oracle access mentioned above.

Acknowledgments. This research has been partially supported by the Swedish Foundation for Strategic Research (SSF) under grant ID24-0087, ZENITH Research and Leadership Career Development Fund, and the German Federal Ministry of Research, Technology and Space (BMFTR) 6GEM+ Transfer Hub under the Grants 16KIS2412 and 16KISS005. The training of the DNNs was enabled by resources provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS).

Disclosure of Interests. The authors declare no competing interests.

References

1. Agresti, A.: Categorical Data Analysis. Wiley, 3rd edn. (2013)
2. Aguilar-Melchor, C., et al.: Hamming quasi-cyclic (HQC) (Nov 2017), <https://hal.science/hal-01946880>, submission to the NIST post quantum standardization process. 2017
3. Alani, M.M.: Neuro-cryptanalysis of DES and Triple-DES. In: Huang, T., Zeng, Z., Li, C., Leung, C.S. (eds.) Neural Information Processing – ICONIP 2012. pp. 637–646. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)
4. Aragon, N., et al.: BIKE: Bit flipping key encapsulation (2022)
5. Baksi, A.: Machine learning-assisted differential distinguishers for lightweight ciphers. In: Classical and Physical Security of Symmetric Key Cryptographic Algorithms, pp. 141–162. Springer (2022)
6. Barker, W.C., Barker, E.B.: Recommendation for the Triple Data Encryption Algorithm (TDEA) Block Cipher. NIST Special Publication 800-67 Rev. 2, National Institute of Standards and Technology (Nov 2017). <https://doi.org/10.6028/NIST.SP.800-67r2>, <https://doi.org/10.6028/NIST.SP.800-67r2>, withdrawn Jan 1, 2024
7. Bernstein, D.J., et al.: ChaCha, a variant of Salsa20. In: Workshop record of SASC. vol. 8, pp. 3–5 (2008)
8. Bindel, N., Brendel, J., Fischlin, M., Goncalves, B., Stebila, D.: Hybrid key encapsulation mechanisms and authenticated key exchange. In: Ding, J., Steinwandt, R. (eds.) Post-Quantum Cryptography – PQCrypto 2019. pp. 206–226. Springer International Publishing, Cham (2019)

9. Bos, J., et al.: CRYSTALS - Kyber: A CCA-secure module-lattice-based KEM. In: IEEE European Symp. Security and Privacy (EuroS&P). pp. 353–367 (2018). <https://doi.org/10.1109/EuroSP.2018.00032>
10. Chollet, F.: Deep Learning with Python. Manning Publications, 2nd edn. (2021)
11. Cramer, R., Shoup, V.: Design and analysis of practical public-key encryption schemes secure against adaptive chosen ciphertext attack. *SIAM Journal on Computing* **33**(1), 167–226 (2003)
12. Duchi, J.: Lecture notes on statistics and information theory (2023)
13. Dworkin, M.: Recommendation for block cipher modes of operation: Methods and techniques. NIST Special Publication 800-38A, National Institute of Standards and Technology, Gaithersburg, MD (2001). <https://doi.org/10.6028/NIST.SP.800-38A>, <https://csrc.nist.gov/publications/detail/sp/800-38a/final>
14. European Commission: A coordinated implementation roadmap for the transition to post-quantum cryptography. Tech. rep., NIS Cooperation Group (2025), <https://digital-strategy.ec.europa.eu/en/library/coordinated-implementation-roadmap-transition-post-quantum-cryptography>
15. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. In: Springer Annual international cryptology conference. pp. 537–554 (1999)
16. Giacon, F., Heuer, F., Poettering, B.: KEM combiners. In: Abdalla, M., Dahab, R. (eds.) *Public-Key Cryptography – PKC 2018*. pp. 190–218. Springer International Publishing, Cham (2018)
17. Gohr, A.: Improving attacks on round-reduced Speck32/64 using deep learning. In: Boldyreva, A., Micciancio, D. (eds.) *Advances in Cryptology – CRYPTO 2019*. pp. 150–179. Springer International Publishing, Cham (2019)
18. Goldreich, O.: Foundations of cryptography, basic tools. Cambridge University Press (2001)
19. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press (2016), <http://www.deeplearningbook.org>
20. Katz, J., Lindell, Y.: Introduction to Modern Cryptography. CRC Press, Boca Raton, FL, 3rd edn. (2020)
21. Kim, B.D., et al.: Cryptanalysis via machine learning based information theoretic metrics (2025), <https://arxiv.org/abs/2501.15076>
22. Merkle, R.C.: Secure communications over insecure channels. *Communications of the ACM* **21**(4), 294–299 (1978)
23. Moriarty, K., Kaliski, B., Jonsson, J., Rusch, A.: PKCS #1: RSA Cryptography Specifications Version 2.2. RFC 8017, Internet Engineering Task Force (Nov 2016). <https://doi.org/10.17487/RFC8017>, <https://www.rfc-editor.org/rfc/rfc8017.txt>
24. Mosca, M., Piani, M.: Quantum threat timeline report 2022. Tech. rep., Global Risk Institute (2022), <https://globalriskinstitute.org/publication/2022-quantum-threat-timeline-report/>
25. National Institute of Standards and Technology: FIPS PUB 46-3: Data Encryption Standard (DES). FIPS Publication 46-3, National Institute of Standards and Technology, Gaithersburg, MD (Oct 1999), withdrawn May 19, 2005
26. National Institute of Standards and Technology: Post-quantum cryptography. <https://www.nist.gov/pqcrypto> (2016–2024), accessed: 2026-02-01
27. National Institute of Standards and Technology: Module-Lattice-Based Digital Signature Standard. Tech. Rep. FIPS 204, National Institute of Standards and Technology (August 2024). <https://doi.org/10.6028/NIST.FIPS.204>

28. National Institute of Standards and Technology: Module-Lattice-Based Key-Encapsulation Mechanism Standard. Tech. Rep. FIPS 203, National Institute of Standards and Technology (August 2024). <https://doi.org/10.6028/NIST.FIPS.203>
29. National Institute of Standards and Technology: Stateless Hash-Based Digital Signature Standard. Tech. Rep. FIPS 205, National Institute of Standards and Technology (August 2024). <https://doi.org/10.6028/NIST.FIPS.205>
30. National Security Agency: COMMERCIAL SOLUTIONS for CLASSIFIED (CSfC), data-at-rest capability package v5.0. Standard, National Security Agency, Fort Meade, MD (2020), <https://www.nsa.gov/Portals/75/documents/resources/everyone/csfc/capability-packages/Data-at-Rest%20Capability%20Package%20v5.0.pdf?ver=u2-HEakuVfDs1-sonI-Now%3D%3D>, accessed: 2026-01-29
31. NIST: NIST releases first 3 finalized post-quantum encryption standards (2024), <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards> [Accessed: 2026-03-04]
32. Rivest, R.L.: Cryptography and machine learning. In: Advances in Cryptology – ASIACRYPT 1991. pp. 427–439. Springer (1991)
33. Rivest, R.L., Shamir, A., Adleman, L.: A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM* **21**(2), 120–126 (1978)
34. Robbins, H., Monro, S.: A stochastic approximation method. *The annals of mathematical statistics* pp. 400–407 (1951)
35. Rukhin, A., et al.: A statistical test suite for random and pseudorandom number generators for cryptographic applications. Tech. Rep. SP800-22 Rev. 1, National Institute of Standards and Technology, Gaithersburg, MD (2010), DOI: 10.6028/NIST.SP.800-22r1a
36. Schneier, B.: Applied cryptography: protocols, algorithms, and source code in C. John Wiley & Sons (2007)
37. Shannon, C.E.: Communication theory of secrecy systems. *The Bell System Technical Journal* **28**(4), 656–715 (1949)
38. Shor, P.W.: Algorithms for quantum computation: discrete logarithms and factoring. In: Proc. 35th Annual Symp. Foundations of Computer Science (FOCS). pp. 124–134. IEEE (1994)
39. Team, T.N.P.: NIST PQC standardization process | HQC announced as a 4th round selection (2025), <https://www.nist.gov/news-events/news/2025/03/nist-pqc-standardization-process-hqc-announced-4th-round-selection> [Accessed: 2026-03-04]
40. Volpe, E., Gauthier-Umaña, V.: Analyzing IND-CPA security of HQC codes using k-nearest neighbors. *IEEE Access* **13**, 184122–184132 (2025). <https://doi.org/10.1109/ACCESS.2025.3623939>